InnoveX' 25

Project Guidelines

Hackathon Dates: 14th August 2025 – 1st September 2025

Mid-Point Review: 29th August 2025

Code Auto-Documenter using Generative AI

Problem Statement

Developers often skip or delay documentation, leading to codebases that are hard to onboard, maintain, and audit. Existing tools generate basic docstrings but struggle with context, multi-file relationships, and explaining design decisions. There is a need for an AI-driven system that understands code structure, intent, and usage to automatically create clear, consistent, and maintainable documentation.

Objective:

Build a system that parses source code, understands its structure and behavior, and generates high-quality documentation including docstrings, module and API references, architecture summaries, and usage examples—continuously kept in sync with the code.

Expected Features

Core Features:

- Multi-language support for at least two programming languages (e.g., Python, JavaScript/TypeScript).

- AST-based parsing to extract symbols: modules, classes, functions, parameters, return types, exceptions.

- LLM-powered generation of docstrings following style guides (e.g., Google/Numpy/Sphinx/JSDoc).

- Module-level README/overview generation summarizing responsibilities and dependencies.

- CLI and Git hook to auto-update docs on commit (pre-commit) or PR.

- Output in Markdown and HTML; optional PDF.

Advanced Features (Optional):

- Cross-file call graph and dependency analysis to produce architecture diagrams.

- Example synthesis: generate runnable code snippets and unit tests from function signatures.

- Change-aware updates: only regenerate docs for impacted symbols; diff-aware summaries in PRs.

- Inline comments improvement suggestions and TODO extraction with priority tags.

- Natural-language Q&A over the repository (RAG) with semantic search on symbols.

- Multi-repo monorepo support; framework-specific adapters (FastAPI, React, Django, Spring).

- Privacy mode with on-device or self-hosted models.

Deliverables

Mid-Point Review (29 August):

1. Working prototype for one language (e.g., Python) generating docstrings and a module README.

2. Basic CLI that scans a small repo and writes Markdown outputs.

3. Demonstration on at least one open-source project with before/after examples.

4. Initial evaluation metrics (readability, coverage %, hallucination checks).

Final Submission (1 September):

- Multi-language support (min. 2 languages) with style-configurable docstrings.

- End-to-end pipeline: parse → analyze → generate → validate → publish (Markdown/HTML, site-ready).

- Integration with CI (GitHub Actions/GitLab CI) and optional PR comment bot.

- Architecture and dependency summaries with optional diagram export (Mermaid/PlantUML).

- Evaluation report with benchmarks on quality, coverage, latency, and cost.

- Documentation, source code, and presentation slides; optional demo video.

Suggested Tech Stack

Code Parsing & Analysis:

- Tree-sitter, LibCST/ast (Python), TypeScript Compiler API, JDT/JavaParser.
- Static analysis tools (pylint, mypy, eslint) for types and contracts.

Generative AI & Retrieval:

- LLMs: OpenAI GPT, Llama, Mistral; prompt templates and function-calling.
- Embeddings + Vector DB: FAISS, Chroma, PostgreSQL pgvector for symbol search.
- Guardrails: JSON schema validation, hallucination checks, unit test validation.

Documentation & Publishing:

- Markdown, Sphinx/MkDocs (Material theme), JSDoc/TypeDoc, Doxygen.
- Mermaid/PlantUML for diagrams; Graphviz for call graphs.

DevOps & Integration:

- Python (Typer/Click) or Node.js (Commander) for CLI.
- Git hooks (pre-commit), GitHub Actions/GitLab CI; Docker for packaging.

Frontend (optional):

- React/Next.js dashboard to preview diffs, search symbols, and run Q&A over the repo.

Input & Output

Input: Source code repository (local path or Git URL), optional config (language, style guide, privacy mode).
Output: Markdown/HTML site (API reference, module overviews), updated inline docstrings, diagrams (Mermaid/PlantUML), and an optional PDF bundle.

Output Example:

Scenario: A FastAPI microservice with 12 endpoints and 18 utility functions.
Input to System: Repository URL + config.yaml specifying Google-style docstrings and MkDocs site.
System Processing:
- Parse symbols and build call graph.
- Generate/refresh docstrings and endpoint summaries with request/response schemas.
- Create architecture.md with Mermaid diagrams and a quickstart example.
- Run CI to publish docs to GitHub Pages.

Output:
- docs/api/users/create_user.md (parameters, returns, errors, example request).
- docs/architecture.md (Mermaid diagram and dependency list).
- Inline Google-style docstrings added to 95% of functions.
- PR comment with diff of docs and coverage report (92% coverage).